МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №3
по курсу «ПГП и ТПП»

Классификация и кластеризация изображений на GPU

Выполнил: Кудрявов Л.В.
Группа: М8О-406Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

1. **Цель работы.** Научиться использовать GPU для классификации и кластеризации изображений. Использование константной памяти и одномерной сетки нитей. Формат изображений тот же, что в лабораторной работе 2; в результирующем изображении на месте альфа-канала записывается номер класса, к которому отнесён пиксель. Если пиксель можно отнести к нескольким классам, выбирается класс с наименьшим номером.
2. **Вариант 4. Метод спектрального угла.** Входные данные: путь к исходному изображению, путь к результату, число классов nc, затем nc строк: количество пикселей выборки np_j и np_j пар координат пикселей выборки. Ограничения: $nc \le 32$, $np_j \le 2^{19}$, $w \cdot h \le 4 \cdot 10^8$. Класс пикселя p определяется выражением $jc = \text{argmax}_j [p^T \cdot avg_j / |avg_j|]$, где avg_j — вектор средних по выборке класса j.

Программное и аппаратное обеспечение

Вычисления выполнялись в облачной среде Google Colab с графическим ускорителем NVIDIA. Характеристики графического процессора приведены в таблице 1. Значения даны для NVIDIA Tesla T4 (compute capability 7.5, та же архитектура, что у GeForce GTX 1650 проверяющей системы). Характеристики соответствуют стандартной конфигурации Google Colab с ускорителем T4.

Таблица 1 — Характеристики оборудования

Параметр	Значение
Графический процессор	NVIDIA Tesla T4
Compute capability	7.5
Графическая (глобальная) память	15 360 МиБ (≈ 15 ГБ), GDDR6
Разделяемая память на блок	48 КБ (64 КБ на мультипроцессор)
Константная память	64 КБ
Регистров на блок	65 536
Максимум нитей в блоке	1024 (размеры блока до 1024 × 1024 × 64)
Максимальный размер сетки	$2^{31} - 1 \times 65\,535 \times 65\,535$ блоков
Число мультипроцессоров	40
Размер варпа	32 нити
Центральный процессор	Intel(R) Xeon(R) CPU @ 2.00GHz, 2 логических ядра
Оперативная память	≈ 12 ГБ
Дисковая память	≈ 100 ГБ

Проверяющая система курса: GeForce GTX 1650 (compute capability 7.5, 4 ГБ глобальной памяти, 48 КБ разделяемой памяти на блок, 65 536 регистров на блок), nvcc 12.0, g++ 13.3.0, Ubuntu 24.04.1 LTS.

Программное обеспечение: операционная система Ubuntu 24.04 (среда выполнения Google Colab), компилятор nvcc из состава CUDA Toolkit 13.0 (V13.0.88), компилятор g++ 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1), Python 3 с библиотеками NumPy, pandas, matplotlib и Pillow (генерация тестов, конвертер изображений, независимая проверка результатов, таблицы). Среда разработки: ноутбуки Jupyter в Google Colab, текстовый редактор. Сдаваемые программы собираются командой `nvcc -arch=sm_75 -Werror cross-execution-space-call -lm`, профилирующие — `nvcc -O2 -arch=sm_75`. Версия nvcc проверяющей системы (12.0) отличается от использованной, но программы применяют только базовые возможности CUDA.

Метод решения

Каждый класс j задаётся выборкой пикселей. Вектор средних цвета класса вычисляется по формуле $avg_j = (1 / np_j)\cdot\Sigma\ ps_i^j$, где ps_i^j — вектор (r, g, b) i-го пикселя выборки. Суммы целочисленные и вычисляются точно, деление выполняется один раз. Затем векторы нормируются: $u_j = avg_j / |avg_j|$.

Для пикселя p выбирается класс с максимальным значением скалярного произведения $p^T\cdot u_j$. Так как |p| одинаково для всех классов, это эквивалентно выбору класса с наименьшим спектральным углом между вектором цвета пикселя и вектором среднего класса (максимальным косинусом). Метод учитывает направление вектора цвета (оттенок), но не его длину (яркость). При равенстве значений выбирается класс с меньшим номером: классы перебираются по возрастанию номера, а сравнение строгое (>). Если вектор среднего нулевой, оценка класса принимается равной нулю.

Нормированные векторы u_j (до 32 классов, 768 байт) копируются в константную память `c_unit`: все нити читают одни и те же данные в одном порядке, что соответствует широковещательному доступу кэша констант. Нить обрабатывает пиксель, используется одномерная сетка фиксированного размера (`<<<256, 256>>>`) с циклом по пикселям с шагом, равным числу нитей. Вычисления проводятся в `double`. Сложность O(w·h·nc). Обработка ведётся «на месте»: номер класса записывается в альфа-канал того же буфера.

Использованные источники: Kruse F.A. et al. The Spectral Image Processing System (SIPS) — interactive visualization and analysis of imaging spectrometer data // Remote Sensing of Environment, 1993; NVIDIA CUDA C++ Programming Guide (Constant Memory); лекции курса (классификация).

Описание программы

Программа находится в файле `lab3.cu`.

- `c_unit[32][3]` — нормированные векторы средних классов в константной памяти (`__constant__ double`).
- `classify(img, n, nc)` — ядро классификации (листинг ниже).
- `main` — чтение путей, изображения и описаний классов с проверками (число классов 1…32, размер выборки, координаты внутри изображения),

вычисление средних, копирование в константную память (`cudaMemcpyToSymbol`), запуск ядра, запись результата. Любая ошибка — сообщение `ERROR: …` в stderr и код возврата 0.

```
__constant__ double c_unit[MAX_CLASSES][3];

__global__ void classify(uchar4 *img, long long n, int nc) {
    long long idx = (long long)blockIdx.x * blockDim.x + threadIdx.x;
    long long offset = (long long)blockDim.x * gridDim.x;
    while (idx < n) {
        uchar4 p = img[idx];
        int best = 0;
        double best_val = 0.0;
        for (int j = 0; j < nc; j++) {
            double val = p.x * c_unit[j][0] + p.y * c_unit[j][1] + p.z *
c_unit[j][2];
            if (j == 0 || val > best_val) { best_val = val; best = j; }
        }
        p.w = (unsigned char)best;     // номер класса в альфа-канал
        img[idx] = p;
        idx += offset;
    }
}
```

Результаты

1. Проверка корректности

Таблица 2 — Тесты программы

Тест	Ожидаемый результат	Результат
Пример из условия: изображение 3 × 3, 2 класса	номера классов 1 0 1 / 1 1 1 / 1 0 0	совпадает
Пример из условия: изображение 8 × 8, 5 классов	совпадение всех 64 пикселей	совпадает
50 случайных изображений (до 200 × 200, до 32 классов)	совпадение с независимой реализацией на NumPy	совпадает
Три одинаковых класса	всем пикселям присвоен класс 0 (наименьший номер)	совпадает
Класс с нулевым вектором средних	ошибки нет, оценка класса равна нулю	совпадает
Пиксель выборки вне изображения; nc = 0; пустой ввод	ERROR в stderr, код возврата 0	совпадает

Порядок координат в выборке определён по примеру 8 × 8: первое число — столбец (x), второе — строка (y); такой порядок даёт полное совпадение, обратный — 47 расхождений из 64.

При замерах результат GPU сравнивался с CPU-реализацией (те же вычисления в `double`): число пикселей с различным классом равно 0 для всех конфигураций и размеров.

2. Скриншоты

На рисунке 1 слева показано синтетическое изображение 480 × 300 с шумом и выборками шести классов (небо, трава, земля, крыша, стена, солнце), справа — результат классификации, где каждый класс закрашен своим цветом. Из-за плохого

разрешения, картинки в Google Colab не отображаются. На рисунке 1 изображены примеры ожидаемого результата.

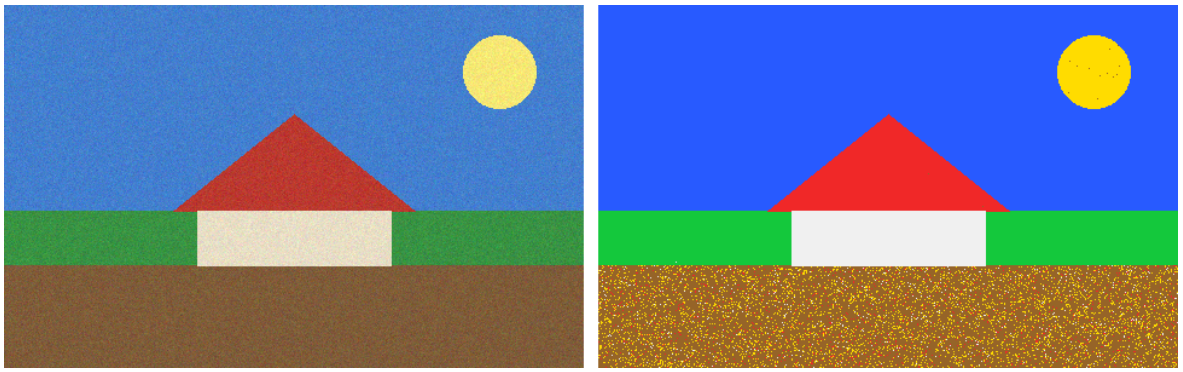


Рисунок 1 — Исходное изображение (слева) и результат классификации методом спектрального угла (справа)

Видно особенность метода: он различает оттенок, но не яркость. Однородные по оттенку области (небо, трава, крыша, стена, солнце) классифицируются чисто, а коричневая область земли, оттенок которой близок к оттенкам красной крыши и жёлтого солнца, содержит вкрапления соседних классов из-за шума.

3. Время работы ядра при разных конфигурациях запуска

Замеры выполнены профилирующей программой `lab3_bench.cu`: ядро то же, что в `lab3.cu`, изображение и 8 случайных классов генерируются внутри программы, сборка `nvcc -O2 -arch=sm_75`. Время ядра измерялось событиями `cudaEvent` и усреднялось по 5 запускам после прогрева (если первый запуск длился больше 1 с, выполнялся один запуск). Сетка одномерная, конфигурация записана как `<<<blocks, threads>>>`; сдаваемая версия использует `<<<256, 256>>>`. Число классов nc = 8.

Таблица 3 — Время работы ядра, мс

Конфигурация	640 × 480	4096 × 4096	20000 × 20000
<<<1, 32>>>	18,25	578,40	13 858,56
<<<1, 1024>>>	6,130	208,41	5 023,23
<<<16, 256>>>	0,427	19,28	310,49
<<<256, 256>>>	0,172	9,159	137,42
<<<1024, 256>>>	0,193	8,509	128,22
<<<1024, 1024>>>	0,179	9,056	128,89

Для изображения 20000 × 20000 время ядра уменьшается с 13 858,56 мс (`<<<1, 32>>>`, один варп) до 128,22 мс для `<<<1024, 256>>>`, то есть в 108 раз. Один блок из 1024 нитей (5 023,23 мс) использует один мультипроцессор из 40. Конфигурация `<<<16, 256>>>` задействует 16 мультипроцессоров и работает в 2,4 раза дольше лучшей, что соответствует отношению числа мультипроцессоров 40/16 = 2,5. Начиная с 256 блоков время практически не меняется (137,42–128,89 мс): устройство загружено полностью.

Ядро обрабатывает 3,1 млрд пикселей в секунду и при этом читает и записывает по 4 байта на пиксель, то есть использует около 25 ГБ/с, что намного меньше пропускной способности памяти T4 (≈ 320 ГБ/с). Следовательно, ядро ограничено не памятью, а вычислениями: на каждый пиксель приходится nc = 8 скалярных произведений в типе `double`, а производительность `double` на Tesla T4 в 32 раза ниже, чем `float`.

4. Сравнение с CPU

Для сравнения в той же программе выполняется последовательная версия на CPU (один поток, Intel Xeon 2,00 ГГц): те же вычисления в `double` и то же правило выбора класса. Время CPU — среднее по 5 проходам для изображений до 10^6 пикселей, для больших — один проход. Время GPU с копированием включает копирование изображения на видеокарту и результата обратно (таблица 4, рисунок 2).

Таблица 4 — Сравнение CPU и GPU

Размер изображения	CPU, мс	GPU, ядро, мс	GPU, с копированием, мс	Ускорение, раз
640 × 480	9,871	0,217	1,921	5,14
4096 × 4096	632,22	10,07	71,53	8,84
20000 × 20000	15 074,75	138,18	1 598,14	9,43

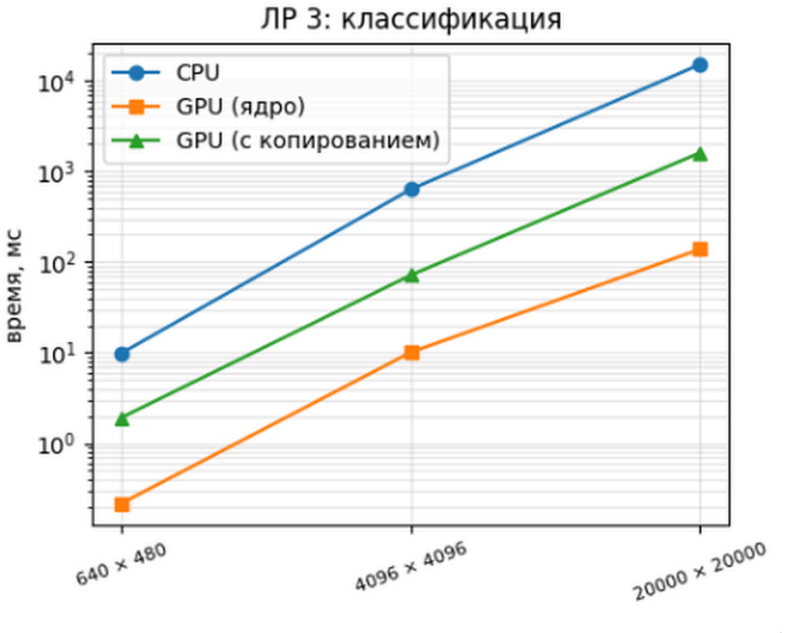


Рисунок 2 — Время классификации на CPU и GPU

Ядро быстрее CPU в 45, 63 и 109 раз; с учётом копирования ускорение составляет 5,14, 8,84 и 9,43 раза. Ускорение растёт с размером изображения, так как постоянные накладные расходы (запуск ядра, копирование в константную память, выделение памяти) распределяются на большее число пикселей, а копирование составляет 91% полного времени при размере 20000 × 20000. Вычислительная нагрузка

на пиксель здесь выше, чем в ЛР 1, поэтому GPU выигрывает у CPU даже с учётом копирования.

Выводы

Классификация по спектральному углу применяется при анализе многоспектральных и гиперспектральных снимков (картографирование земной поверхности, определение типов растительности и материалов), в задачах сегментации изображений по цвету. Инвариантность к яркости полезна при неравномерном освещении, но, как показывает рисунок 1, делает метод непригодным для различения областей, отличающихся только яркостью.

Задача хорошо подходит для GPU: пиксели независимы, данные классов малы (до 768 байт) и помещаются в константную память, где доступны всем нитям без повторных обращений к глобальной памяти. Сложность программирования невысока; основные трудности — разбор формата входных данных (порядок координат определён по примеру), строгий порядок выбора класса при равенстве оценок, обработка ошибочного ввода без аварийного завершения.

Замеры на Tesla T4 показали: конфигурация запуска меняет время ядра в 108 раз (от одного варпа до полной загрузки устройства), при этом с 256 блоков время не уменьшается; конфигурация на 16 блоков медленнее лучшей в 2,4 раза, как и следует из числа мультипроцессоров (40). Ядро быстрее последовательной версии на CPU в 45–109 раз, а с учётом копирования ускорение составляет 5,14–9,43 раза и растёт с размером изображения. Время ядра ограничено вычислениями в `double`, а не памятью: для ускорения можно было бы использовать `float`, если допустима меньшая точность.